



1. Контекстное окно как рабочая область агента

Когда мы работаем с агентом, нужно перестать воспринимать модель как систему, которая одновременно «знает» всё содержимое нашего проекта. В каждый конкретный момент модель работает только с той информацией, которая попала в её текущее контекстное окно.

Контекстное окно я могу сравнить с рабочим столом разработчика. На диске может находиться огромный проект, Git-история, документация и тысячи исходных файлов, но для решения текущей задачи нам не обязательно одновременно открывать всё это содержимое. Модели тоже необходимо выбрать ту информацию, которая непосредственно относится к текущему вопросу.

В контекст попадает не только сообщение пользователя. Там уже присутствуют системные инструкции Hermes, проектный контекст, часть истории диалога, результаты предыдущих действий, описания доступных инструментов и данные, которые агент получает во время работы. Поэтому заявленный размер контекстного окна модели нельзя воспринимать как объём, полностью доступный только для нашего исходного кода.

Чем дольше продолжается сессия и чем больше информации мы передаём агенту, тем сильнее этот ограниченный ресурс начинает влиять на работу.

2. Токены и реальная стоимость контекста

Размер контекста обычно измеряется в токенах. Токен — это внутренняя единица текста, с которой работает языковая модель. Одно слово может занимать один токен, несколько токенов или быть объединено с соседними элементами текста в зависимости от используемой токенизации.

Для нас гораздо важнее другое: любой переданный код тоже занимает токены. Если я передам модели пять файлов, они займут часть контекста. Если передам пятьсот файлов, объём увеличится уже очень существенно.

Причём входной контекст конкурирует за место с результатом, который модель должна сгенерировать. Мы не можем бесконечно заполнять окно исходными данными, а потом ожидать, что у модели останется неограниченное пространство для анализа и ответа.

В Hermes дополнительное место занимает и инфраструктура самого агента. Например, подключение большого числа инструментов может увеличивать объём передаваемых модели схем. В Hermes даже существует механизм Tool Search, который позволяет не держать схемы всех подключённых MCP- и plugin-инструментов в активном контексте постоянно, а открывать их по необходимости. Само наличие такого механизма хорошо показывает, что контекст является ограниченным вычислительным ресурсом, которым приходится управлять.

3. Контекст и память — это не одно и то же

Теперь нам необходимо разделить два понятия, которые очень часто смешивают: контекст и память.

Контекст — это информация, непосредственно доступная модели сейчас. Память — это механизм сохранения информации, которую можно использовать в другой момент или в другой сессии.

Например, если мы десять сообщений обсуждаем миграцию проекта, содержание предыдущих сообщений находится в текущем контексте. Для этого модели не требуется обращаться к долговременной памяти.

Если же мы закрыли эту сессию и через несколько дней начали новую, старый диалог уже не обязан находиться в активном контексте. Тогда понадобится какой-то механизм восстановления нужной информации.

В Hermes для постоянной памяти используются, в частности, `MEMORY.md` и `USER.md`. Они содержат компактный набор важных сведений и загружаются в системный контекст в начале новой сессии. При этом такая память специально ограничена по объёму и не предназначена для хранения всей истории проекта или тысяч файлов исходного кода.

Следовательно, память не решает проблему большого репозитория. Мы не должны пытаться сохранить исходный код проекта в `MEMORY.md`. Для этого нужен другой механизм.

4. Почему нельзя просто сделать один бесконечный диалог

Может возникнуть логичный вопрос: если история текущей сессии уже находится в контексте, почему просто не работать в одном разговоре месяцами?

Технически длинную сессию поддерживать можно, но постепенно она становится всё тяжелее. Накапливаются старые сообщения, промежуточные результаты, вывод инструментов и решения задач, которые уже давно закончены.

Hermes использует механизм управления контекстом и компрессии, когда разговор приближается к доступному лимиту модели. Встроенный context engine отслеживает использование токенов и может выполнять сжатие контекста, чтобы длинная сессия продолжала работать.

Но компрессия не означает магического расширения окна. Часть подробностей приходится сворачивать в более компактное представление. Чем больше старой информации мы пытаемся сохранить, тем сильнее приходится решать, что важно, а что можно сократить.

Поэтому полезная стратегия заключается не в бесконечном накоплении контекста, а в правильном разделении информации. Постоянные правила проекта должны находиться в проектном контексте, действительно важные долгосрочные сведения — в памяти, а конкретные материалы для текущей задачи должны подбираться непосредственно перед её выполнением.

5. Long-context стратегия

Теперь мы можем понять смысл long-context моделей. Такие модели способны принять очень большой объём информации за один запрос.

На первый взгляд кажется, что это полностью решает проблему большой кодовой базы. Если модель поддерживает сотни тысяч или даже миллион токенов, можно попытаться передать ей значительную часть проекта сразу.

Но большое контекстное окно не означает, что нужно обязательно его заполнять.

Представьте вопрос: «Где в проекте используется устаревший метод авторизации?» Если у нас есть семь тысяч файлов, передавать все семь тысяч только ради поиска нескольких релевантных мест будет нерационально.

Даже если всё физически помещается, модель получает огромное количество нерелевантного материала. Увеличивается стоимость обработки, растёт время запроса и усложняется поиск действительно важных фрагментов.

Поэтому long-context стратегия хорошо работает тогда, когда большой объём материала действительно необходимо анализировать совместно. Например, когда изменения затрагивают несколько тесно связанных модулей и нам важно одновременно видеть их контракты, тесты и вызывающий код.

Большое окно — это дополнительная возможность, а не замена качественному отбору контекста.

6. Chunking — разделение большой базы знаний

Чтобы не передавать большой документ или кодовую базу целиком, информацию разделяют на части. Этот процесс называется chunking, а отдельная часть обычно называется chunk.

В самом простом варианте большой текст можно делить через определённое количество символов или токенов. Но для исходного кода такое механическое разрезание не всегда удачно.

Представьте, что функция начинается в одном chunk, а её завершение попадает в следующий. Мы формально получили два допустимых фрагмента, но каждый из них отдельно хуже отражает смысл программы.

Поэтому при работе с кодовой базой желательно учитывать структуру исходного кода. Естественной единицей может становиться функция, класс, метод, модуль или логически связанный участок файла.

Хороший chunk должен быть достаточно маленьким, чтобы поиск мог точно определить его релевантность, но достаточно содержательным, чтобы модель могла понять найденный фрагмент.

Если chunks слишком большие, мы снова начинаем передавать много ненужного текста. Если они слишком маленькие, теряются связи и контекст.

7. Векторное представление и векторный индекс

После того как кодовая база разбита на chunks, нам необходимо научиться находить среди них фрагменты, связанные с запросом пользователя.

Один вариант — обычный текстовый поиск. Если пользователь спрашивает про `calculate_price`, мы можем найти точное вхождение этого имени.

Но запросы не всегда совпадают с кодом буквально. Пользователь может написать «где рассчитывается итоговая стоимость заказа», а в исходном коде вообще не будет этой точной фразы.

Для семантического поиска используются `embeddings`. Модель преобразует содержимое `chunk` в числовой вектор, который отражает определённые смысловые свойства текста.

Запрос пользователя преобразуется аналогичным образом. Затем система сравнивает вектор запроса с векторами сохранённых фрагментов и выбирает наиболее близкие.

Набор таких представлений и механизм быстрого поиска по ним формируют векторный индекс.

Таким образом, индекс не хранит «ответы ИИ». Он помогает определить, какие части исходных данных наиболее вероятно относятся к текущему запросу.

8. Retrieval-Augmented Generation

Теперь мы можем собрать предыдущие понятия в RAG — Retrieval-Augmented Generation.

Смысл RAG заключается в том, что модель не должна получать всю базу знаний заранее. Перед генерацией ответа сначала выполняется `retrieval`, то есть поиск релевантной информации. После этого найденные материалы добавляются к запросу, и только затем модель формирует результат.

Представим вопрос: «Какие модули придётся изменить при переходе на новую версию платёжного API?» Поисковый слой сначала находит релевантные части репозитория. Это могут быть клиент платёжного сервиса, конфигурация, обработчики платежей и связанные тесты.

Затем именно эти фрагменты попадают в контекст модели. Модель получает не семь тысяч файлов проекта, а небольшой набор материалов, наиболее тесно связанных с поставленной задачей.

Поэтому RAG нужно понимать не как новый тип языковой модели. Это архитектура подачи информации уже существующей модели.

Для агентной системы RAG особенно полезен, потому что позволяет получать дополнительный контекст непосредственно в момент выполнения задачи. Архитектура Hermes поддерживает добавление динамического контекста перед обращением к модели через `plugin`-механизмы; документация прямо рассматривает этот слой как подходящий в том числе для RAG-интеграций.

9. RAG для кодовой базы отличается от памяти Hermes

Здесь особенно важно не смешать несколько разных механизмов Hermes.

Встроенный `session_search` позволяет Hermes искать по прошлым разговорам. Он использует полнотекстовый индекс SQLite FTS5 и возвращает реальные сообщения из предыдущих сессий. Это `retrieval`, но это не векторный RAG по исходному коду проекта.

`MEMORY.md` и `USER.md` решают другую задачу. Они сохраняют небольшой набор устойчивых сведений между сессиями.

`AGENTS.md`, с которым мы работали на прошлом занятии, задаёт постоянные правила и архитектурный контекст проекта. Hermes автоматически добавляет такой файл в системный контекст соответствующей рабочей области.

А RAG по кодовой базе будет решать совершенно другую задачу: искать конкретные фрагменты исходного проекта, связанные с текущим техническим запросом.

Поэтому правильно построенная агентная система может использовать все эти механизмы одновременно. Они не заменяют друг друга, потому что работают с разными типами информации и имеют разный жизненный цикл.

10. Как должен выглядеть управляемый контекст агента

Теперь я хочу собрать всю тему в одну общую модель.

Когда агент начинает работу в Hermes, ему уже необходим небольшой устойчивый слой контекста. Там находятся правила системы, проектные инструкции и важная постоянная информация.

Дальше появляется текущая задача. Для её выполнения агенту не требуется автоматически загружать весь репозиторий.

Вместо этого система должна определить, какая дополнительная информация действительно нужна. Если вопрос касается конкретной части кодовой базы, поисковый слой находит соответствующие chunks и передаёт их модели как дополнительный контекст текущего запроса.

Если найденной информации недостаточно, можно выполнить ещё один поиск и расширить контекст. Если часть информации больше не нужна, её необязательно постоянно переносить во все последующие запросы.

Именно поэтому управление памятью и контекстом я рассматриваю как задачу архитектуры, а не просто как выбор модели с самым большим количеством токенов.

Хорошая агентная система не стремится дать модели максимум информации. Она стремится в нужный момент предоставить достаточную и релевантную информацию.

На следующей практике мы как раз переведём эту идею в работающую схему для Hermes: возьмём кодовую базу, разделим материалы на chunks, построим индекс, научимся искать релевантные фрагменты и передавать найденный контекст агенту вместо загрузки всего проекта целиком.